Versión 1.4

Paradigmas de Programación

Paradigma Orientado a Objetos

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

"La programación orientada a objetos es un método de implementación en el que los programas se organizan como colecciones cooperativas de objetos, cada uno de los cuales representa una instancia de alguna clase, y cuyas clases son, todas ellas, miembros de una jerarquía de clases unidas mediante relaciones de herencia." Grady Booch

Inquietudes Elementales

¿Qué es un Objeto?
Es una entidad que tiene un conjunto de responsabilidades, encapsula un estado interno y posee identidad. Este conjunto de responsabilidades se implementa a través de métodos los cuales determinan el comportamiento del objeto. El estado interno se implementa a través de un conjunto de propiedades o atributos encapsulados, puntualmente, está dado por el conjunto de valores de sus propiedades en un instante determinado. La identidad es la característica que permite distinguir a un objeto entre todos los demás.

¿Qué es una Clase?
Se la puede considerar como *el molde para un tipo determinado de objetos*. Un objeto es una instancia de la clase a la que pertenece, y todo objeto es instancia de alguna clase.

¿Qué es una Propiedad?
Una propiedad (atributo) representa una característica que ha sido identificada durante la etapa de modelado, para la clase en cuestión.
Las propiedades tienen un nombre y un tipo de dato asociado, a su vez, cada instancia de la clase mantendrá un valor para cada propiedad. El conjunto de valores definidos en cada una de las propiedades, en un momento dado, determinarán el **estado interno** de la instancia.
Tomando la clase Persona como ejemplo, dependiendo del problema que se esté modelando, algunas propiedades que podrían identificarse serían: apellido, dirección, peso, altura, sueldo, etc.
Cada propiedad cuenta con la posibilidad de definirle dos **descriptores de acceso**, uno denominado **Get**, el cual se ejecutará al momento de leer el valor contenido o calculado por la propiedad, y otro denominado **Set**, que se ejecutará al momento de intentar escribir (asignar) un valor en ella.
Cuando una propiedad cuenta con ambos descriptores de acceso implica que dicha propiedad es de lectura y escritura; si solo cuenta con el descriptor Get es de solo lectura; si solo cuenta con el descriptor Set es de solo escritura.
La propiedad que requiera mantener su estado, deberá contar con un espacio en memoria para almacenar el valor asignado, a dicho espacio se lo denomina **campo**. El campo debe declararse con alcance privado para impedir el acceso directo desde el exterior, hecho que infringiría el concepto de encapsulamiento al exponer el estado interno del objeto sin pasar por la lógica definida en cada propiedad.
Cuando para una propiedad no se declara un campo de manera explícita, automáticamente C# le genera uno. A partir de net 10, el campo generado puede ser referenciado dentro del cuerpo de la propiedad, a través de la palabra clave **field**.
Por convención, el identificador de un campo comienza con un guion bajo.
En C# existe un tercer descriptor de acceso denominado **init**, el cual solo permitirá asignarle un valor a la propiedad dentro de un *constructor* o un *inicializador de objetos*. Una vez creado el objeto, dicha propiedad se vuelve inmutable.

Definición de Propiedades

- **Campo para propiedad FechaNac**

```
private DateTime _fechaNac;
```

- **Propiedad FechaNac de lectura y escritura**

```
public DateTime FechaNac
{
```

```
   get { return this._fechaNac; }
   set {
         if (value <= DateTime.Today)
              this._fechaNac = value;
       }
}
```

- **Propiedad Edad de solo lectura**

```
public int Edad
{
   get {
      int anios = DateTime.Today.Year - this.FechaNac.Year;
      if (this.FechaNac.AddYears(anios) > DateTime.Today)
         anios--;
      return anios;
   }
}
```

Definición de Propiedades con Expression-bodied Accessors

- **Campo para propiedad FechaNac**

```
private DateTime _fechaNac;

public DateTime FechaNac
{
   get => this._fechaNac;
   set => this._fechaNac = (value <= DateTime.Today? value : this._fechaNac);
}
```

Definición de Propiedades sin Campo Explícito

- **Con Restricciones**

```
public DateTime FechaNac
{
   get;
   set {
         if (value <= DateTime.Today)
              field = value;
       }
}
```

- **Sin Restricciones**

```
public DateTime FechaNac {get; set;}
```

¿Qué es un Método?

Un método es una función o procedimiento que ha sido identificado durante la etapa de modelado, para la clase en cuestión.

Representa una acción que podrá ser solicitada a cada instancia de la clase. Los métodos pueden modificar el estado interno del objeto e interactuar con objetos de otras clases.

Los métodos tienen una firma, la cual está constituida por el nombre del método, sus parámetros y el tipo de retorno. El conjunto de métodos definidos en una clase define el comportamiento de cada una de sus instancias.
Tomando la clase Persona como ejemplo, dependiendo del problema que se esté modelando, algunos métodos que podrían identificarse serían: correr, saltar, escribir, pagar, etc.

¿Qué es un Programa en POO?
Un programa es una colección de objetos cooperantes, esta cooperación se lleva a cabo a través de mensajes.

¿Qué es un Mensaje?
Consiste en el nombre de una operación junto con los argumentos necesarios. Es el medio por el cual los objetos se comunican entre sí con el fin de lograr una acción.
El camino que se sigue para dar respuesta a un mensaje es el siguiente:

1. El objeto recibe un mensaje.
2. Se busca el mensaje en la clase del objeto receptor.
3. Si el mensaje es encontrado, se ejecuta el método correspondiente.
4. Si el mensaje no fue encontrado, se lo busca en la superclase del objeto receptor. Si no se lo encuentra, se lo buscara en la superclase de su superclase, y así sucesivamente.
5. Si finalmente el mensaje no fue encontrado, se da un mensaje de error ya que el objeto no fue capaz de entender el mensaje.

Pilares del Paradigma Orientado a Objetos

Los siguientes conceptos son considerados los pilares fundamentales de este paradigma:

- Abstracción.
- Encapsulamiento.
- Modularidad.
- Polimorfismo.
- Jerarquía y Herencia.

Al mencionar Fundamentales, se pretende señalar que "un modelo que carezca de cualquiera de estos elementos no puede ser catalogado como un modelo orientado a objetos". ***[Booch]***

Nota: la introducción al paradigma y sus características fueron tratadas en la Unidad 1.

Abstracción
La abstracción captura el comportamiento y las características esenciales de un objeto, para un contexto determinado; es uno de los caminos fundamentales mediante los cuales los seres humanos enfrentan la complejidad.
Es clave en el proceso de análisis y diseño orientado a objetos, ya que mediante ella es posible definir al conjunto de clases que modelarán la realidad (el problema a enfrentar).
"La abstracción denota las características esenciales de un objeto que lo distinguen de todos los demás tipos de objetos y proporciona así fronteras conceptuales bien definidas respecto a la perspectiva del observador.". [Booch].

Encapsulamiento
Un objeto pone a disposición de los demás objetos todo aquello que es capaz de realizar, pero nunca informa o les permite saber cómo lleva a cabo dichas tareas.
Esta característica es de suma importancia, asegura que la implementación en los objetos clientes es independiente de la implementación en el objeto servidor. Por lo tanto, los algoritmos contenidos en los métodos del objeto servidor pueden ser modificados (perfeccionados) con tranquilidad sabiendo que no implicarán cambios en los objetos clientes, siempre que no se haya modificado la interfaz del servidor. Pero

no solo se trata de impedir que el cliente conozca los detalles del servidor, sino algo más importante aún: *evitar que el cliente se vea forzado a conocerlos*.
La abstracción y el encapsulamiento son conceptos complementarios: la abstracción se concentra en la visión exterior del objeto, mientras que el encapsulamiento evita que los objetos clientes accedan a su visión interna donde el comportamiento de la abstracción es implementado.
"*El Encapsulamiento es el proceso de esconder todos los detalles de un objeto que no contribuyen a sus características esenciales*". [Booch].

Modularidad
Este concepto no es propio de la orientación a objetos. Se denomina modularidad a la propiedad que permite subdividir una aplicación compleja en partes más pequeñas y generalmente más sencillas (llamadas módulos), cada una de las cuales debe ser tan independiente como sea posible de la aplicación en sí y de las partes restantes. Estos módulos interactúan entre sí y, de ser necesario, pueden ser transportados a otros proyectos. La reutilización es una de las bases del modelo orientado a objetos.
"La modularidad es la propiedad que tiene un sistema que ha sido descompuesto en un conjunto de módulos cohesivos y débilmente acoplados." [Booch]

Polimorfismo
Es la habilidad de dos o más objetos pertenecientes a clases diferentes pero relacionadas, de responder a un mensaje que lleva la misma firma, cada uno según su propia implementación. En consecuencia, un objeto no necesita conocer el tipo del objeto receptor, es suficiente con saber que este es capaz de entenderlo (responderlo).
Ejemplificando, una colección podría contener objetos de diferentes clases, y la invocación de un comportamiento sobre cualquiera de ellos producirá el comportamiento correcto para el tipo de objeto referenciado.

Jerarquía y Herencia
La jerarquía es la posibilidad de realizar un ordenamiento en niveles de aquello que se desea modelar. En la práctica esta jerarquía se ve reflejada por la herencia. Las clases no están aisladas, sino que se relacionan entre sí formando una jerarquía de clasificación. Los objetos heredan las propiedades y el comportamiento de todas las clases a las que pertenecen. La herencia organiza y facilita el polimorfismo y el encapsulamiento, permitiendo a sus instancias ser definidas y creadas como tipos especializados de clases preexistentes. Estas pueden compartir (y extender) su comportamiento sin necesidad de volver a implementarlo.
"La jerarquía es una clasificación u ordenación de abstracciones." [Booch]

Binding Dinámico
Es el mecanismo por el cual el enlace entre el mensaje y la implementación concreta del método se realiza en tiempo de ejecución (no compilación). Sin Binding Dinámico no sería posible implementar el polimorfismo.

Modelado en POO

Al diseñar o modelar un sistema, en el contexto del paradigma, se pretende desarrollar clases donde las responsabilidades estén bien asignadas y la delegación sea adecuada.
Cuando se habla de **Responsabilidad** se hace referencia al conjunto de tareas que deberá llevar acabo la clase, mientras que al hablar de **Delegar** se hace referencia a canalizar en otras clases las tareas que no le son propias.
Responsabilidad denota una obligación de ella para proporcionar un determinado comportamiento.
Delegar es el acto por el cual una clase cede una operación a otra clase, para que ésta la realice en nombre de la primera.
Una métrica importante es la **Cohesión**, mide que tan bien se han asignado las responsabilidades a una clase: es Alta cuando la clase se orienta a una responsabilidad bien definida y acotada, mientras que es Baja cuando la clase mezcla responsabilidades dispares.

Por lo tanto, la cohesión de una clase refleja el nivel de afinidad entre las responsabilidades que concentra y no la cantidad de responsabilidades asignadas.
Otra métrica relevante es el **Acoplamiento**, mide el grado de dependencia entre clases: es Bajo cuando las clases se relacionan poco entre sí, en consecuencia, los cambios en una clase no implican o requieren modificaciones en otras clases. Es Alto, cuando las clases están fuertemente vinculadas generando así un modelo difícil de mantener.
¿Cómo delegar y mantener un bajo acoplamiento? Una alternativa, a través de referencia a interfaces o clases abstractas y no directamente a clases concretas. Lo relevante es cómo se delega y no cuánto se delega.
En conclusión, se pretende lograr un modelo con una alta cohesión y bajo acoplamiento. Encontrar este equilibrio suele requerir de mucho análisis y experiencia: si una clase no delega nada, evitaría el acoplamiento pero podría estar acumulando responsabilidades no afines y perdería cohesión. Si delega mal (apoyándose en implementaciones concretas), quizás gane cohesión pero generará un alto acoplamiento.
Las métricas Cohesión y Acoplamiento permitirán determinar si las decisiones de diseño, asignación de responsabilidades y delegación de tareas, fueron correctas.

Conceptos Relacionados

Visibilidad
Para que un objeto “A“ pueda enviar un mensaje a otro objeto “B”, la propiedad o método (del objeto “B”) que es referenciado en el mensaje debe ser visible para la clase del objeto “A”.
La visibilidad determina desde que lugares es posible acceder a cada miembro (propiedad o método) de una clase. Este mecanismo permite controlar el encapsulamiento, determinando cuales miembros serán accesibles desde el exterior y cuales permanecerán ocultos.

Grado de Visibilidad	Alcance
Public	La clase, atributo o método puede ser accedido desde cualquier lugar de la solución.
Internal	La clase, el atributo o método puede ser accedido desde cualquier lugar dentro del mismo proyecto.
Protected	La clase, atributo o método puede ser accedido dentro de la misma clase y desde las clases que hereden de ella.
Private	La clase, atributo o método puede ser accedido solo dentro de la clase contenedora.

Constructor
Un constructor es un método especial, se caracteriza por llevar el mismo nombre de la clase en la cual se lo ha definido y a su vez, por no especificar un valor de retorno. Es el método a través del cual la clase construye sus instancias. Allí se definirán los valores de inicialización para cada propiedad de la nueva instancia, y adicionalmente, permitirá incluir tareas que se consideren necesarias realizar en cada instanciación.
Una clase puede contar con múltiples constructores, tantos como se estime conveniente. Un constructor se deberá diferenciar de los restantes de la clase por la cantidad de parámetros definidos y el orden asignado a los tipos de datos asociados.

Constructor	Estado
public Persona ()	Válido
public Persona (int dni, string nombre)	Válido
public Persona (string nombre, int dni)	Válido
public Persona (int legajo, string apellido)	Inválido, la clase ya cuenta con un constructor que recibe dos parámetros y los mismos tipos de datos en idéntica ubicación

Al momento de construir una instancia, la clase determinará el constructor a implementar en función de los parámetros especificados. Un constructor podría no contar con parámetros.
Cuando una clase no cuenta con constructores definidos, los compiladores suelen brindarle un constructor por defecto.
En C# existe una sintaxis denominada **inicializador de objetos** que permite, al momento de crear un objeto, detallar el valor que deberá ser asignado a cada propiedad que se desee tratar sin necesidad de contar con un constructor que acepte ese conjunto de propiedades como argumento. Solo requiere que la clase cuente con un constructor sin parámetros y que las propiedades involucradas dispongan de su descriptor de acceso init o setter accesible.

Clase Persona	Main

internal class Persona
{
public int Dni { get; set; }
public int Legajo { get; set; }
public string Apellido { get; set; }
public string Nombre { get; set; }
 	
public Persona()
{
Dni = 1;
Legajo = 1000;
Apellido = string.Empty;
Nombre = string.Empty;
}
}
	Persona per = new Persona()
	{
	Dni = 1234,
	Apellido = "Garcia"
	};
	
	Pese a no contar con un constructor que recibe como parámetros el dni y el apellido, el objeto es creado con los valores indicados; las propiedades omitidas asumirán los valores especificados en el constructor sin argumentos.

Sobrecarga
En una clase pueden definirse métodos con el mismo nombre siempre que el conjunto de parámetros declarados por ellos se diferencie en alguno de los siguientes aspectos: cantidad, tipo u orden de tipos. Cuando un método presenta este tipo de comportamiento (variantes), se dice que dicho método se encuentra sobrecargado. La sobrecarga brinda flexibilidad, le otorga al usuario la posibilidad de optar por la variante que más se ajuste a su necesidad.
El mismo concepto aplica a los constructores.
A continuación, se presentan cuatro sobrecargas posibles para el método Procesar:

- public void Procesar (int A, string B, long C)
- public void Procesar (int A, string B)
- public void Procesar (long C, string B)
- public void Procesar (string B)

Algunos lenguajes, contemplan inclusive la posibilidad de implementar sobrecarga y a la vez, variar el tipo de dato de retorno.

Redefinición
Tal como se ha definido con anterioridad, cada clase está en condiciones de definir las propiedades y métodos que considere necesarios para cumplir con la funcionalidad para la cual ha sido desarrollada.
Una clase abstracta está habilitada para definir propiedades y métodos concretos, abstractos y virtuales, mientras que una clase no abstracta solo podrá definir concretos y virtuales.
Cuando una propiedad o método fue declarado como **abstracto** implica que su definición no incluye una implementación, en consecuencia, cada subclase se verá obligada a redefinirlo fijando la implementación más adecuada según su naturaleza.

Cuando una propiedad o método fue declarado como **virtual** implica que su definición incluye una implementación por default, de esta manera, cada subclase tendrá la posibilidad de redefinirlo dando lugar a una implementación diferente a la heredada, pero es de importancia comprender que dicha redefinición no es obligatoria. En caso de optar por no realizar una redefinición, la subclase responderá con la implementación (heredada) establecida en su superclase, la implementación por default.
Una propiedad o método concreto (no abstracto ni virtual), que ha sido heredado no puede ser redefinido.
La instrucción que se debe utilizar para realizar la redefinición es **override**.

Variables y Referencias
Si el lenguaje utilizado aplica Semántica por Referencias (ej.: C#, javacript), las variables de tipo objeto almacenan una referencia al objeto, de este modo, puede presentarse una situación particular: múltiples variables referenciando a un mismo objeto. En este contexto, una asignación entre variables implicará copiar una referencia al objeto apuntado por la variable original.
En lenguajes que aplican Semántica por Valor (ej.: C++), las variables contienen al objeto (como sucede con tipos de datos primitivos) y al realizar una asignación entre variables se realizará una copia del objeto, resultando objetos independientes pero con idéntico estado.

Interfaz de una Clase vs Implementación de una Clase
Cuando se habla sobre la **interfaz** de una clase, se refiere al conjunto de métodos que la clase ofrece para interactuar con sus instancias, definiendo **qué** es posible hacer con un objeto de la clase sin exponer **cómo** lo hace internamente (para no violar el encapsulamiento).
Está dada por el conjunto de firmas de los métodos accesibles (nombre, parámetros y tipos de retorno).
Cuando se habla sobre la **implementación** de una clase, se refiere a la implementación interna de los métodos expuestos en su interfaz. Es decir, está dada por el código interno (algoritmo) del conjunto de métodos públicos, métodos privados y atributos privados. Gracias al encapsulamiento, la implementación puede ser actualizada/mejorada sin afectar a los objetos que hacen uso de las instancias de la clase, siempre y cuando se mantenga inalterable la interfaz de la misma.

Misma Identidad vs Igualdad
Se entiende que dos variables tipo objeto tienen **misma identidad** si hacen referencia al mismo objeto.
Sin embargo, puede ser necesario determinar si dos variables tipo objeto con referencias diferentes, pueden considerarse **iguales**; es decir, si conceptualmente están representando a objetos equivalentes. La igualdad se determina comparando el estado interno de los objetos involucrados según un criterio definido por el desarrollador de la clase.

Nota: en c#, por default, tanto el método Equals como el operador == determinan si ambos objetos tienen la misma identidad.

Propiedades de Clase y Métodos de Clase
Una propiedad de clase es un atributo que pertenece a la propia clase. Son compartidas por todas sus instancias y existe una única copia del valor en memoria, independientemente de cuántos objetos se hayan creado. Algunos usos prácticos: llevar la cuenta de las instancias creadas, definir valores constantes y definir valores de configuración comunes a todos los objetos de la clase.
Un método de clase opera sobre la clase en lugar de hacerlo sobre una instancia concreta, y no tiene acceso a las propiedades de instancia.
Para hacer uso de las propiedades y métodos de clase, no se requiere generar una instancia de la misma, es decir, se acceden directamente a través de la clase.

Nota: en c#, se identifican con la palabra reservada static. Es posible definir una clase completa como static, en este caso no podrá instanciarse ni heredarse. Son clases especiales creadas con la finalidad de funcionar solo como un contenedor de comportamiento, no un molde para generar instancias ni dar herencia.

Clase Abstracta
Es una clase que no ha sido creada para producir instancias de sí misma. Este tipo de clases son creadas con la única finalidad de agrupar comportamiento y estructura interna común a varias subclases.

Interface
Es una entidad que declara una serie de atributos y métodos sin implementación. Es una especie de contrato, donde toda clase que la implemente deberá obligatoriamente implementar todas las funcionalidades establecidas por la interfaz.
Una interfaz, al establecer solo declaraciones, nunca podrá ser instanciada.

Diferencias y semejanzas entre Clases Abstractas e Interfaces		
Habilidad	**Clases Abstractas**	**Interfaces**
Permite generar instancias	No	No
Permite definir alcance Protected	Si	No
Permite definir implementaciones por default	Si	No por definición, pero desde C# 8 es posible.
Posibilita Inyección de dependencias	No	Si
Posibilita aplicar herencia múltiple	No en la gran mayoría de los lenguajes. Una excepción es C++.	Si

Colección
Una colección es una clase que brinda a sus instancias la capacidad de contener un conjunto de objetos. Toda colección permitirá conocer la cantidad de objetos almacenados, agregar y quitar elementos sobre sí misma, entre otras tantas tareas que variarán según el tipo de colección implementada.
Existen diferentes tipos de colecciones, cada una con sus particularidades (ventajas y desventajas) que harán de ella una colección más apropiada frente a las demás para ser implementada en la resolución del problema a tratar.

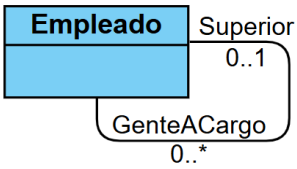
Colección	**Características**
List<T>	Permite acceder a sus elementos por medio de un índice
Queue<T>	Permite manipular sus elementos como una cola de espera es decir, se comporta como FIFO (el primero en entrar es el primero en salir)
Stack<T>	Permite manipular sus elementos como una pila es decir, se comporta como LIFO (el último en entrar es el primero en salir)
Dictionary<TKey,TValue>	Permite almacenar elementos expresados como pares Clave/Valor posibilitando un acceso rápido a través de la Clave. Si se intenta agregar un elemento cuya Clave ya existe, se presentará un error. Permite recuperar un Valor a partir de la Clave. Permite eliminar un elemento a partir de la Clave.
SortedDictionary<TKey,TValue>	Es similar a un Dictionary pero los elementos se mantienen automáticamente ordenados por Clave. El criterio de ordenamiento utilizado es el predeterminado para el tipo de dato correspondiente a la Clave pero es posible, codificación previa, definir un criterio propio.
SortedList<TKey,TValue>	Es similar a un SortedDictionary y proporciona un manejo de memoria más eficiente (utiliza menos memoria) pero para las operaciones de inserción y eliminación brinda un rendimiento menor.

Relaciones entre clases

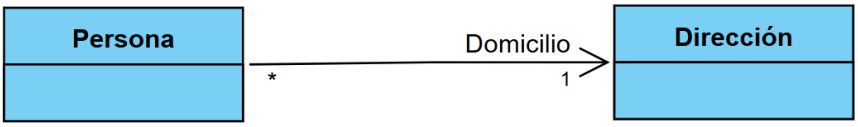
Asociación

Una relación de asociación describe una vinculación entre objetos, estos objetos pueden pertenecer a clases diferentes o a la misma clase (autoasociación).

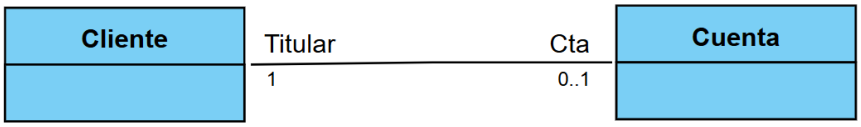
- **Autoasociación:** una clase está asociada a sí misma. Ejemplificando: Un Empleado podrá tener a cargo varios Empleados, uno o ninguno, a su vez todos tendrán un Empleado responsable de ellos o ninguno (ninguno: el propietario de la empresa figuraría sin un superior). Las propiedades que se agregan a Empleado a partir de la relación se denominan GenteACargo y Superior.



- **Unidireccional:** una clase A esta asociada a otra clase B, pero no sucede en sentido inverso. Ejemplificando: una instancia de la clase Persona tendrá asociada una instancia de la clase Dirección, mientras que una instancia Dirección no tendrá asociada una instancia Persona. Es decir, una Dirección no conoce la Persona o Personas que la tienen asociada.



- **Bidireccional:** una clase A esta asociada a una clase B y viceversa. Ejemplificando: un Cliente puede o no tener asociada una CuentaBancaria, y una CuentaBancaria tendrá asociado un Cliente.



Agregación

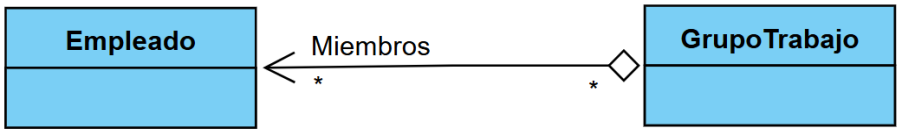
Es un caso especial de asociación donde se pretende aclarar que una de las clases involucradas representa un TODO y la otra una PARTE de ella, donde la parte puede existir independientemente del todo. La parte se generará fuera del Todo, y es pasada a él a través de algún medio (constructor/método).

La Parte tiene existencia fuera del Todo, tiene sentido por sí misma. Incluso, un mismo objeto Parte podría ser "parte" de varios objetos Todo.

Ejemplificando: un Grupo de Trabajo puede estar constituido por varios Empleados, y un Empleado puede formar parte de varios Grupos de Trabajo.

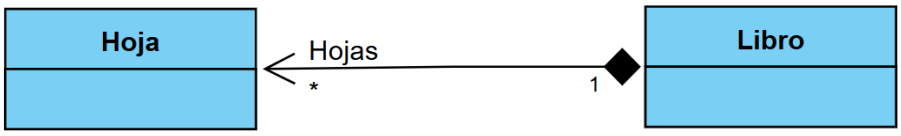
Eliminar un Grupo de Trabajo, no implica eliminar los Empleados.

En el modelo representado, el empleado desconoce a los grupos que pertenece por ser una relación unidireccional.



Composición

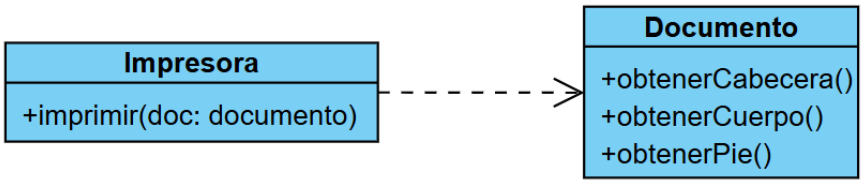
Es un caso especial de asociación donde se pretende aclarar que una de las clases involucradas representa un TODO y la otra una PARTE de ella, donde la Parte no puede existir independientemente del Todo. La Parte se genera dentro del Todo, no puede ser pasada a él desde el exterior.
La Parte, no tiene de existencia fuera del Todo, no tiene sentido por sí misma. Por lo tanto, nunca un mismo objeto Parte podría ser "parte" de varios objetos Todo.
En una relación de composición hay una regla de "no compartir".
Ejemplificando: un Libro estará constituido por varias Hojas, y una Hoja no puede formar parte de otros Libros. Eliminar un Libro, implica la destrucción de sus Hojas.
En el modelo representado, la hoja desconoce el libro al que pertenece por ser una relación unidireccional.
La clase Libro incluiría un método que le permitiría generar sus Hojas.



Característica	Agregación	Composición
Dos o más compuestos pueden compartir un mismo componente	Sí	No
Al destruir el compuesto se destruyen los componentes	No	Sí

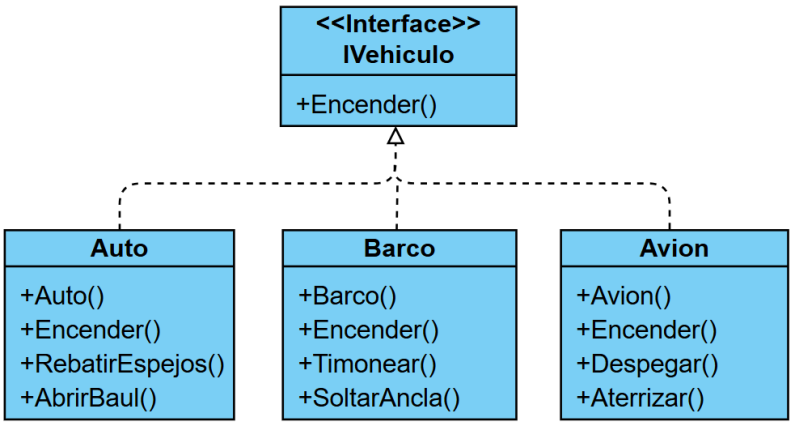
Dependencia

Una dependencia entre clases denota una relación de uso de una sobre otra. Esta situación se presenta cuando un método de la clase A recibe como parámetro un objeto de la clase B o dentro de algún método de la clase A se instancia un objeto de la clase B para desempeñar una tarea determinada. En estas situaciones, se dice que A depende de B, es decir, hace uso de sus servicios.
Por lo cual, en caso de sufrir modificaciones la clase de la cual se depende, es posible que deban introducirse cambios en la clase dependiente.



Realización/Implementación

Una realización denota una relación entre una interfaz y una clase, donde la clase se encuentra obligada a cumplir con el contrato establecido, es decir, deberá implementar el conjunto de propiedades y métodos declarados en la interfaz.



Generalización/Herencia

La generalización es una relación entre una clase más general (denominada superclase o clase padre) y una clase más específica (denominada subclase o clase hija).

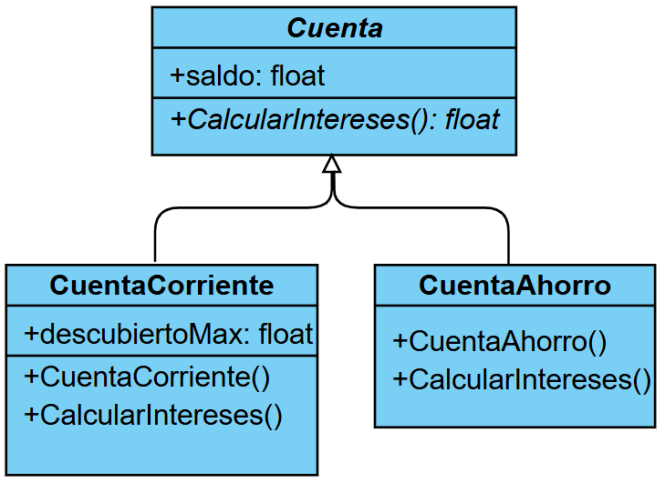
Toda subclase hereda los atributos y métodos de la superclase, y a su vez puede añadir sus propios atributos y métodos.

Las subclases representan un tipo específico de la superclase, por ello, toda instancia de la subclase puede interpretarse a su vez como instancia de la superclase. Ejemplificando, si se cuenta con la superclase Persona y una subclase de ella es la clase Cliente, implica que una instancia de la clase Cliente (más allá de ser un Cliente) no deja de ser una Persona.

Este razonamiento, permite concluir/entender que toda instancia de la subclase puede emplearse en cualquier espacio donde se requiera una instancia de la superclase.

Conceptualmente, una clase puede heredar de múltiples clases (herencia múltiple), aunque en la práctica no siempre es posible de implementar, dependerá del lenguaje utilizado.

Toda clase (concreta o abstracta) puede ser referenciada como superclase. Es decir, una superclase puede ser abstracta pero no es obligatorio, depende del contexto y del modelo que se pretenda desarrollar.



Anexo

Tipo Enumerado (Enum)

Un tipo enumerado o enumeración, es un tipo de dato que define una lista de valores constantes, cada uno de ellos identificado con un nombre. De esta manera, cualquier elemento (variable, propiedad, etc.) declarado con este tipo de dato, estará capacitado para aceptar solo aquellos valores que han sido definidos en la enumeración.

Cada constante definida en la enumeración, tendrá asignado un numero entero. Por defecto, a la primera constante se le asignará el valor 0; las restantes serán numeradas automáticamente incrementando en una unidad el valor de la constante inmediatamente anterior.

En el siguiente ejemplo se define una enumeración denominada *temporada* para la cual solo se admitirán las constantes: primavera, verano, otoño e invierno.

Ejemplo 1	Ejemplo 2	Ejemplo 3

enum Temporada
{
Primavera,
Verano,
Otoño,
Invierno
}
 	
primavera tendrá asociado el valor 0, verano el valor 1, otoño el valor 2 e invierno el valor 3.
	enum Temporada
	{
	Primavera = 10,
	Verano,
	Otoño = 75,
	Invierno
	}
	
	primavera tendrá asociado el valor 10, verano el valor 11, otoño el valor 75 e invierno el valor 76.
		enum Temporada
		{
		Primavera = 10,
		Verano = 30,
		Otoño = 75,
		Invierno = 90
		}
		
		primavera tendrá asociado el valor 10, verano el valor 30, otoño el valor 75 e invierno el valor 90.

Operaciones con Enum

- **Obtener el valor entero asociado a una constante:**

```
int valor = (int)Temporada.Primavera;
```

- **Convertir un valor entero a la constante correspondiente:**

```
int valor = 10;
Temporada estacion = (Temporada)valor;
```

- **Convertir una cadena, ignorando mayúsculas y minúsculas, a la constante correspondiente:**

```
temporada estacion;
if (Enum.TryParse<Temporada>("PrimaVera", true, out estacion))
{
   Console.WriteLine("Conversión exitosa.");
}
else
{
   Console.WriteLine("Atención: conversión fallida.");
}
```

PascalCase y camelCase

Son convenciones de nomenclaturas para los identificadores utilizados en el código fuente.

- **PascalCase** se aplica a los identificadores de clases, propiedades, métodos, constantes y tipos enumerados (incluyendo sus constantes). Consiste en colocar con mayúscula la primera letra de cada palabra que compone al identificador, manteniendo a los restantes caracteres en minúscula. Ej.: ApellidoNombre, ComponerFrase, DomicilioLegal, etc.

- **camelCase** se aplica a los identificadores de variables, parámetros y campos. Consiste en colocar con minúscula la primera letra de la primera palabra que compone al identificador; las demás palabras deben comenzar con una mayúscula y sus restantes caracteres en minúsculas. Ej.: edadMinima, importeNeto, _nombreFantasia, etc.

Base

La palabra clave base se utiliza para acceder a miembros de la clase padre (base) desde una clase hija (subclase), permitiendo reutilizar la lógica previamente definida. No saltea niveles de herencia, es decir, hace referencia a la clase inmediatamente superior. Solo puede ser utilizado para referenciar constructores, propiedades y métodos de instancia, es decir, no estáticos.

```
public class Persona
{
   public int Dni { get; set; }
   public string Apellido { get; set; }
   public Persona()
   {
      this.Dni = 1;
      this.Apellido = string.Empty;
   }
   public Persona(int dni, string apellido)
   {
      this.Dni = dni;
      this.Apellido = apellido;
   }
   public virtual string Describir()
   {
      return "Soy una Persona.";
   }
}

public class Empleado : Persona
{
   private int Legajo { get; set; }
   public Empleado(int dni, int legajo, string apellido) : base(dni, apellido)
   {
      this.Legajo = legajo;
   }
   public override string Describir()
   {
      string mensaje = base.Describir();
      return $"{mensaje} Además soy Empleado.";
   }
}
```

Signo de Pregunta
El signo de pregunta "?" cuenta con múltiples usos posibles, algunos de ellos:

Tipo nullable
Permite indicar que un tipo de valor (int, bool, datetime, etc.) acepta un null, evitando así generar una excepción al recibir un null.

- **Ejemplos:**

```
int? importe = null;
DateTime? fecha = null;
```

Operador ternario
Permite expresar un condicional en una única línea.
condición ? valorSiTrue : valorSiFalse;

- **Ejemplos:**

```
string mensaje = edad >= 18 ? "es adulto" : "es menor de edad";
int premio = cargo == "Gerente" ? 70 : 50;
```

Acceso condicional
Permite acceder a miembros de un objeto solo si el mismo no es null evitando así la necesidad de validar previamente, a través de un condicional, si el objeto en cuestión tiene una referencia null.
Aplica también sobre arreglos.

- **Ejemplos:**

```
Departamento? depto = null;
string? propietario = depto?.ApelNomPropietario;

string[]? apellidos = null;
string? apel = apellidos?[0];
int? cantCaracteres = apellidos?[0].Length;
```

Tupla
Permite retornar un conjunto de valores sin necesidad de definir una clase para ello. Se define encerrando entre paréntesis los tipos de datos que determinan su estructura, separándolos por comas.
En la firma del método, junto a cada tipo de dato retornado, es posible especificar un identificador para facilitar la legibilidad.
Al momento de recuperar la tupla, en caso de no definir un identificador para cada uno de los valores recibidos, se podrá acceder a cada uno de ellos a través del identificador ItemX donde X es la posición (iniciando en 1) que ocupa el valor dentro de la tupla.

- **Ejemplos:**

```
public (int doc, string ape, string nom) ObtenerDatos()
{
   return (250, "Garcia", "Javier");
}

//sin definir identificadores para los valores retornados
(int, string, string) per1 = ObtenerDatos();
Console.WriteLine($"{per1.Item1} - {per1.Item2}, {per1.Item3}");
```

```
//definiendo identificadores para cada valor retornado
(int Dni, string Apellido, string Nombre) per2 = ObtenerDatos();
Console.WriteLine($"{per2.Dni} - {per2.Apellido}, {per2.Nombre}");

//definiendo identificadores para cada valor retornado,
//pero de manera desestructurada (variables sueltas)
(int Dni, string Apellido, string Nombre) = ObtenerDatos();
Console.WriteLine($"{Dni} - {Apellido}, {Nombre}");
```

Struct

Informalmente podría definirse como una clase light. Si el conjunto de propiedades que define no supera los 16 bytes (dependiendo del contexto) brinda un rendimiento superior a una clase con características similares; las comparaciones entre instancias se realizan por valor (solo a través de Equals, no acepta los operadores == ni !=) y no es capaz de implementar herencia desde clases ni structs. Por lo demás, puede implementar propiedades, métodos, constructores, interfaces, etc. de manera similar a las clases.

- **Ejemplos:**

```
public struct Coordenada
{
   public float Latitud { get; init; } // 4 bytes
   public float Longitud { get; init; } // 4 bytes
}
```

Record

La palabra clave Record se utiliza con clases y estructuras (struct).

Con respecto a las clases, las define como **inmutables**, generando automáticamente las propiedades definidas con sintaxis posicional, con descriptores de acceso get e init. De esta manera, simplifica la creación de clases en las que se requiere inmutabilidad como por ejemplo DTOs.

Se permite generar clones (copias) con datos modificados a través de with.

Otra característica de importancia es que las instancias de una record class se comparan por valor (tanto con Equals como con == y !=), mientras que las clases no record se comparan por referencia.

Una record class puede volverse mutable si se le adicionan propiedades con el descriptor de acceso set.

Al definir un struct como record se accede a la posibilidad de comparar instancias entre si con los operadores == y !=, y puede definirse como inmutable si se agrega readonly a su definición.

- **Ejemplos:**

```
internal record class Persona(int Dni, string Apellido, string Nombre);

Persona p1 = new Persona(120, "Gomez", "Joaquin");
Persona p2;
p2 = p1 with {Apellido = "Rosales"}; //p1 mantiene los valores originales
```

	Class	Record Class	Struct	Record Struct
Soporta Herencia	Sí	Sí, solo de otros records.	No	No
Soporta with	No	Sí	Sí	Sí
Comparación con Equals	Por Referencia	Por valor	Por valor	Por valor
Comparación con == y !=	Por Referencia	Por valor	No aceptados	Por valor
Inmutabilidad	Mutable, salvo que se aclare readonly	Inmutable al implementar sintaxis posicional, salvo que se definan expresamente descriptores de acceso set.	Mutable, salvo que se aclare readonly	Mutable, salvo que se aclare readonly
Soporta Sintaxis Posicional	No	Sí	No	Sí

Sintaxis Posicional

La sintaxis posicional permite definir para un record tipo clase o struct: las propiedades, su constructor y su deconstructor todo en una sola línea de código, basándose en la posición de los argumentos.

- **Ejemplos:**

```
internal record class Persona(int Dni, string Apellido, string Nombre);

Persona p1 = new Persona(120, "Gomez", "Joaquin");
Console.WriteLine($"ApelNom: {p1.Apellido}, {p1.Nombre}");
```

Operaciones en Listas

Operación	Características
Add	Permite agregar un elemento a la lista. Ejemplo: `personas.Add(persoA);`
All	Retorna True si **todos** los elementos cumplen con la condición. Ejemplo: `bool todosCumplen = personas.All(p => p.Edad > 18);`
Any	Retorna True si existe **algún** elemento que cumpla con la condición. Ejemplo: `bool existeAlguno = personas.Any(p => p.Edad > 18);`
Clear	Permite eliminar todos los elementos de la lista. Ejemplo: `personas.Clear();`

Contains	Retorna True si el elemento existe en la lista. Ejemplo: `bool existe = personas.Contains(persoA);`
FirstOrDefault	Retorna el primer elemento que cumple con la condición y un null en caso de no encontrar ninguno. Ejemplo: `Persona? buscado = personas.FirstOrDefault(p => p.Edad >= 18);`
Max	Retorna el máximo valor que alcanza la propiedad indicada. Genera una excepción si la lista está vacía. Ejemplo: `int edadMaxima = personas.Max(p => p.Edad);`
Min	Retorna el mínimo valor que alcanza la propiedad indicada. Genera una excepción si la lista está vacía. Ejemplo: `int edadMinima = personas.Min(p => p.Edad);`
OrderBy	Permite obtener una copia de la lista con sus elementos ordenados en forma ascendente según la propiedad indicada. Ejemplo: `List<Persona> ordenados = personas.OrderBy(p => p.Apellido).ToList();`
Remove	Permite eliminar la primera ocurrencia del elemento. Ejemplo: `personas.Remove(persoA);`
RemoveAll	Permite eliminar todos los elementos que cumplan con la condición indicada. Ejemplo: `personas.RemoveAll(p => p.Edad > 18);`

Select	Permite obtener una copia de la lista que resulta aplicar para cada elemento de la lista original la operación indicada.
	Ejemplo:
	`List<Persona> conUnoMas = personas.Select(p =>`
	`new Persona(p.Dni, p.Apellido, p.Nombre, p.Edad + 1)).ToList();`
	
	`List<int> soloEdadesConUnoMas = personas.Select(p =>p.Edad + 1).ToList();`

Sum	Retorna la sumatoria de los valores de la propiedad indicada. Ejemplo: `int edadTotal = personas.Sum(p => p.Edad);`
Where	Permite obtener una copia de la lista con los elementos que cumplen la condición indicada. Ejemplo: `List<Persona> soloMayores = personas.Where(p => p.Edad >= 18).ToList();`

Contenido

Inquietudes Elementales ____ *1*
Pilares del Paradigma Orientado a Objetos ____ *3*
Abstracción ____ 3
Encapsulamiento ____ 3
Modularidad ____ 4
Polimorfismo ____ 4
Jerarquía y Herencia ____ 4
Binding Dinámico ____ 4
Modelado en POO ____ *4*
Conceptos Relacionados ____ *5*
Visibilidad ____ 5
Constructor ____ 5
Sobrecarga ____ 6
Redefinición ____ 6
Variables y Referencias ____ 7
Interfaz de una Clase vs Implementación de una Clase ____ 7
Misma Identidad vs Igualdad ____ 7
Propiedades de Clase y Métodos de Clase ____ 7
Clase Abstracta ____ 8
Interface ____ 8
Colección ____ 8
Relaciones entre clases ____ *9*
Asociación ____ 9
Agregación ____ 9
Composición ____ 10
Dependencia ____ 10
Generalización/Herencia ____ 11
Anexo ____ *12*
Tipo Enumerado (Enum) ____ 12
PascalCase y camelCase ____ 12
Base ____ 13
Signo de Pregunta ____ 14
Tupla ____ 14
Struct ____ 15
Record ____ 15
Sintaxis Posicional ____ 16
Operaciones en Listas ____ 16